# Week 10 Capstone Project

## Fine-Tuned UrduLish Banking Assistant: Your Own LLM for Fintech

*LLM Fine-Tuning • LoRA / QLoRA • DPO Alignment • LLM Evaluation • Quantization • vLLM Serving • Guardrails • RAG + Fine-Tune Hybrid*

**Duration:** 5 Days

---

## 📌 Scenario

Your new client is Zarya Digital Bank (fictional), a fast-growing Pakistani digital bank with 2 million app users. Their customer support team receives thousands of messages every day on the app, WhatsApp and helpline — mostly in UrduLish:

- "Mera Raast transfer fail ho gaya lekin paise kat gaye"
- "Debit card block karna hai, gum ho gaya hai"
- "ATM se paise nahi niklay lekin balance kam ho gaya"
- "Mujhe call aayi thi bank se, OTP maang rahe thay — kya yeh theek hai?"

They tried a GPT-based chatbot, but the bank has three serious problems with it:

- Data privacy: the State Bank's rules and the bank's own policy do not allow customer conversations to be sent to a foreign API
- Cost: at their message volume, API costs are too high
- Quality: the generic model sounds robotic in UrduLish, does not follow the bank's tone, and sometimes invents policies

Your task is to build the bank's own LLM: take an open-source model, fine-tune it on banking conversations in UrduLish, align it to be safe and helpful, compress it so it runs on the bank's own server, and deploy it as a production assistant that answers customers, classifies complaints, creates tickets and protects customers from fraud.

Weeks 7–9 used LLMs through APIs. This week students go inside the model: they train it, align it, evaluate it, optimise it and serve it themselves.

> ⚠️ **Data rule:** Use only synthetic or public data. No real customer names, CNICs, account numbers or card numbers anywhere in the dataset, code or demo.

The final product should be deployable for an actual client.

## Technologies Expected

The project should expose students to the complete modern LLM engineering stack.

| Area | Tools |
|---|---|
| Base Models | Qwen 2.5 / Qwen 3 (1.5B–7B), Llama 3.x (3B–8B), Gemma, Mistral |
| Fine-Tuning | Hugging Face Transformers, PEFT, TRL, Unsloth (recommended for free Colab GPU) |
| Methods | LoRA, QLoRA, SFT, DPO (or ORPO / KTO) |
| Synthetic Data | Teacher LLM (GPT / Claude / Gemini), Distilabel (optional), Argilla for human review |

| Area | Tools |
|---|---|
| Experiment Tracking | Weights & Biases or MLflow |
| Evaluation | LLM-as-a-judge, DeepEval / Ragas, lm-evaluation-harness, human evaluation |
| Quantization | bitsandbytes, GGUF (llama.cpp), AWQ / GPTQ |
| Serving | vLLM (recommended), Ollama, llama.cpp server, TGI |
| RAG | ChromaDB / pgvector, BGE-M3 or multilingual-e5 embeddings |
| Guardrails | Llama Guard, NeMo Guardrails or Guardrails AI, Presidio for PII |
| Backend / Frontend | FastAPI (OpenAI-compatible API); Streamlit, Gradio or Next.js |
| Model Hub | Hugging Face Hub (private repo) |
| Compute | Google Colab / Kaggle T4 GPU (free tier is enough with QLoRA + Unsloth) |
| Deployment | Docker, Hugging Face Spaces, RunPod / Modal (optional), any VPS |

## Project Objectives

Build a production-ready, self-hosted banking LLM capable of:

- ✅ Answering banking questions naturally in UrduLish, English and Roman Urdu
- ✅ Following the bank's tone, policies and escalation rules
- ✅ Classifying intent and complaint category with structured JSON output
- ✅ Creating support tickets automatically
- ✅ Detecting fraud / scam situations and warning customers
- ✅ Refusing to ask for or reveal PINs, OTPs, passwords or full card numbers
- ✅ Combining fine-tuning (style + behaviour) with RAG (latest policies and fees)
- ✅ Beating the base model and matching a large API model on the bank's tasks
- ✅ Running on a single affordable GPU or CPU after quantization
- ✅ Serving through an OpenAI-compatible API
- ✅ Measurable evaluation, safety testing and cost analysis

## Task Categories the Model Must Handle

| Category | Examples |
|---|---|
| Account & Balance | Account opening, statement request, balance query, limits, dormant accounts |
| Transfers & Payments | Raast / IBFT failures, wrong transfer, pending transactions, bill payments |
| Cards | Block / unblock card, lost card, international usage, ATM cash not dispensed |

| Category | Examples |
|---|---|
| Loans & Products | Eligibility questions, installment queries, product comparison |
| Complaints | Classify, apologise, collect details, create ticket, give TAT (turnaround time) |
| Fraud & Scams | Fake calls asking for OTP, phishing links, SIM swap, suspicious transactions |
| Out of Scope | Politics, medical, investment tips, other banks' internal matters — polite refusal |

---

## Week 10 — Day 1: Problem Framing & Dataset Engineering

> 📌 **Scenario:** Fine-tuning is 80% data. A small model trained on 3,000 excellent examples beats a big model trained on 30,000 messy ones. Today you build the dataset that will define how the bank's AI thinks and talks.

### Task 1 — Fine-Tune vs Prompt vs RAG — Decision Document

Research and write a short decision document comparing:

- Prompt engineering only
- RAG only
- Fine-tuning only
- Fine-tuning + RAG (hybrid)

Compare on quality, cost, privacy, latency, effort and how easy it is to update. Justify why this client needs the hybrid approach.

### Task 2 — Data Specification

Define the dataset before generating it:

- Chat format (system / user / assistant) and chat template of your chosen base model
- Task mix: % of FAQ, complaint, fraud, refusal, multi-turn, JSON tasks
- Language mix: UrduLish, Roman Urdu, English, Urdu script (optional)
- Bank persona and tone guide (polite, short, respectful — "Ji", "aap")
- JSON schema for intent + ticket output

### Task 3 — Synthetic Data Generation

Generate at least 3,000 high-quality training examples:

- Write 100+ hand-crafted "seed" examples
- Use a teacher LLM to expand seeds into varied conversations (different personas, typos, slang, angry customers)
- At least 20% multi-turn conversations
- At least 300 safety / refusal / fraud examples
- At least 300 structured-output (JSON) examples

Document the generation prompts and costs.

### Task 4 — Data Quality & Cleaning

Implement:

- Deduplication (exact + near-duplicate with embeddings)

- Language and length filters
- PII scan with Presidio — no real-looking CNIC / card / account numbers
- Human review of at least 200 random samples in Argilla or a spreadsheet
- Fix or remove incorrect answers and policy hallucinations

Report how many examples were removed at each step.

### Task 5 — Splits, Test Set & Data Card

Prepare:

- Train / validation split
- A separate "golden" test set of 200 examples written or verified by humans, never used for training
- A dataset card (sources, size, task mix, language mix, known limitations)
- Upload the dataset to a private Hugging Face repo

---

## Week 10 — Day 2: Baselines & Supervised Fine-Tuning (LoRA / QLoRA)

> 📌 **Scenario:** Before training anything, measure how good the existing options are. Then fine-tune a small open-source model and prove it is better for this bank.

### Task 1 — Baselines

Run the golden test set through:

- Base open-source model (no fine-tuning) with a good system prompt
- Base model + few-shot examples
- A large API model (GPT / Claude / Gemini) with the same prompt

Save all outputs — these are the numbers to beat.

### Task 2 — Model Selection

Choose 2 candidate base models (e.g. Qwen 2.5 3B and Llama 3.2 3B). Compare:

- Size and GPU memory needed
- Licence (can the bank use it commercially?)
- UrduLish / Roman Urdu ability in baseline tests
- Context length
- Tokenizer efficiency for Roman Urdu (tokens per sentence)

### Task 3 — LoRA / QLoRA Theory

Explain in your own words with diagrams:

- What LoRA does and why it needs so little memory
- Rank (r), alpha and target modules
- QLoRA: 4-bit base model + LoRA adapters
- Full fine-tuning vs LoRA — when to use which
- Chat templates and why loss is computed only on assistant tokens

### Task 4 — Supervised Fine-Tuning (SFT)

Fine-tune both candidate models with QLoRA (Unsloth + TRL recommended):

- Train on assistant responses only
- Log training and validation loss in W&B / MLflow

- Try at least 3 configurations (rank 8 / 16 / 32, learning rate, epochs)
- Save checkpoints and LoRA adapters

Show loss curves and explain any overfitting you see.

### Task 5 — First Comparison

Run the golden test set on the fine-tuned models and create a comparison table:

- Base vs few-shot vs fine-tuned vs API model
- JSON validity rate
- Intent classification accuracy
- Average response length
- 10 side-by-side examples showing the difference

Pick the best model to continue with.

---

## Week 10 — Day 3: Evaluation Framework & Preference Alignment (DPO)

> 📌 **Scenario:** "It looks better" is not good enough for a bank. Today you build a serious evaluation framework, and then use preference tuning to teach the model which answers the bank actually prefers — and which it must never give.

### Task 1 — Evaluation Harness

Build an automated evaluation pipeline that scores:

- Task accuracy: intent and complaint category (accuracy, macro F1)
- Structured output: JSON validity and field accuracy
- Answer quality: LLM-as-a-judge with a written rubric (correctness, helpfulness, tone, UrduLish fluency, conciseness) on a 1–5 scale
- Policy faithfulness: does it invent fees, limits or rules?
- Safety: refusal rate on harmful requests, false refusal rate on normal requests

Validate the LLM judge: compare its scores with human scores on 50 examples and report agreement.

### Task 2 — Human Evaluation

Run a blind evaluation:

- 3+ evaluators
- 50 questions, answers from 3 models shown in random order without names
- Evaluators pick the best answer and rate tone and fluency

Report win rates (e.g. fine-tuned vs API model).

### Task 3 — Preference Dataset

Create at least 500 preference pairs (prompt, chosen, rejected):

- Chosen: polite, correct, safe, short, follows escalation rules
- Rejected: robotic, too long, invents policy, asks for OTP, gives investment advice, rude

Use the SFT model's own weak outputs as "rejected" examples where possible.

### Task 4 — DPO Training

Train the SFT model with DPO (or ORPO / KTO):

- Explain how DPO differs from SFT and from RLHF with a reward model

- Tune beta and learning rate
- Track reward margins and chosen / rejected log-probabilities

Re-run the full evaluation: SFT vs SFT + DPO. Did safety and tone improve? Did accuracy drop?

### Task 5 — Regression & Forgetting Checks

Check that fine-tuning did not break the model:

- General ability check on a small benchmark (e.g. a subset of MMLU or HellaSwag with lm-evaluation-harness)
- English quality check
- Behaviour on completely new topics not in training

Document any catastrophic forgetting and how you reduced it (e.g. mixing in general data).

---

## Week 10 — Day 4: Quantization, Serving, RAG Hybrid & Guardrails

> 📌 **Scenario:** A model that needs an A100 is useless to the client. Today you shrink the model, serve it fast on affordable hardware, connect it to the bank's live policy documents, and wrap it in guardrails.

### Task 1 — Merge & Quantize

Implement:

- Merge LoRA adapters into the base model
- Create at least 3 versions: FP16, 8-bit, 4-bit (GGUF Q4_K_M and / or AWQ)
- Benchmark each: model size, GPU / RAM usage, tokens per second, time to first token
- Re-run the golden test set on every version — how much quality is lost?

Recommend the best version for production and justify it.

### Task 2 — High-Performance Serving

Serve the chosen model with vLLM (or Ollama / llama.cpp for CPU):

- OpenAI-compatible endpoint (/v1/chat/completions)
- Streaming responses
- Load test with 10, 50 and 100 concurrent users (e.g. Locust or a simple async script)
- Measure throughput, p50 / p95 latency and GPU memory

Explain continuous batching and the KV cache in simple words.

### Task 3 — RAG + Fine-Tune Hybrid

Fine-tuning teaches style and behaviour; facts change. Build RAG for:

- Fees and charges schedule
- Transaction limits
- Product terms
- Complaint turnaround times

Use multilingual embeddings (BGE-M3 or multilingual-e5). Show a case where the fee changes: update the document only, without retraining, and the model answers correctly. Compare fine-tuned + RAG vs fine-tuned only vs base + RAG.

### Task 4 — Guardrails

Add input and output guardrails:

- Block requests for PIN / OTP / password and never output full card or account numbers
- Detect scam scenarios and reply with a fixed safety warning
- Prompt-injection detection
- Off-topic and harmful content filtering (Llama Guard or NeMo Guardrails)
- PII masking in logs

Example scam reply: "Zarya Bank kabhi bhi call ya SMS par aap ka OTP ya PIN nahi maangta. Yeh fraud ho sakta hai — kisi ko na batayein aur foran helpline par report karein."

### Task 5 — Ticketing & Tools

Use the model's structured output to:

- Create a complaint ticket (category, priority, summary, customer-reported details) in PostgreSQL
- Return a ticket number and TAT to the customer
- Escalate to a human agent for high-priority cases (fraud, large amount, very angry customer)

Build a simple FastAPI backend that connects model → guardrails → RAG → tools.

---

## Week 10 — Day 5 (Capstone): Deployment, Cost Analysis, Presentation & Client Handover

> 📌 **Scenario:** Today you show the bank's CTO and Head of Customer Experience that their own model is safe, good and cheaper than the API — and hand it over.

### Task 1 — Production Deployment

Deploy:

- Dockerized vLLM / Ollama model server
- FastAPI backend with guardrails, RAG and ticketing
- Customer chat UI and a support-agent dashboard (tickets, escalations, conversation history)
- Environment variables and secrets
- Health checks, logging and basic CI

Push the final model (adapter + quantized version) to a private Hugging Face repo with versioning.

### Task 2 — Monitoring & Continuous Improvement

Implement and document:

- Logging of prompts, responses, latency and guardrail triggers (with PII masked)
- Thumbs up / down feedback in the UI
- A pipeline that turns bad conversations into new training / preference data
- Retraining schedule and model version rollback plan

### Task 3 — Cost & ROI Analysis

Compare for 50,000 messages per day:

- API model cost per month
- Self-hosted cost per month (GPU server or CPU)

- Latency and quality of each
- Break-even point

Present a one-page recommendation for the bank's management.

### Task 4 — Stakeholder Demonstration

Prepare a 10-minute live demo showing:

- Base model vs your fine-tuned model on the same UrduLish question
- Failed transfer complaint → ticket created with number and TAT
- Scam call scenario → safety warning
- Customer asking for OTP / card number → refusal
- Fee question answered from RAG, then updated without retraining
- Evaluation dashboard: accuracy, judge scores, human win rate, safety results
- Speed and cost comparison
- Support-agent dashboard with escalations

### Task 5 — Documentation & Future Enhancements

Write:

- Model Card (base model, data, training, evaluation, limitations, intended use, risks)
- Dataset Card
- Training and evaluation reproducibility guide
- API documentation
- Operations guide for the bank's IT team

Propose future enhancements such as: Urdu-script and voice support, distillation into an even smaller model for mobile, function-calling to core banking APIs, multilingual support (Punjabi, Sindhi, Pashto), continuous DPO from live feedback, and fraud-pattern detection across conversations.

---

## Evaluation Rubric

| Criteria | Weight |
|---|---|
| Dataset engineering & data quality | 20% |
| Baselines & SFT with LoRA / QLoRA | 15% |
| Evaluation framework (automated + human) | 15% |
| DPO alignment & safety behaviour | 10% |
| Quantization, serving & load testing | 15% |
| RAG hybrid, guardrails & ticketing | 10% |
| Deployment, cost analysis, documentation & demo | 15% |

**Minimum targets:** Fine-tuned model beats the base model on every metric; intent accuracy ≥ 90%; JSON validity ≥ 98%; 100% refusal on OTP / PIN requests; human win rate vs base ≥ 70%; 4-bit model loses less than 3% quality; p95 latency under 3 seconds.

## Final Submission Requirements

The final submission should include:

- ✅ Decision document: prompting vs RAG vs fine-tuning vs hybrid
- ✅ Synthetic training dataset (3,000+ examples) with a dataset card
- ✅ Human-verified golden test set (200 examples)
- ✅ Baseline results for base, few-shot and API models
- ✅ SFT training notebooks, configs and loss curves (W&B / MLflow)
- ✅ Preference dataset (500+ pairs) and DPO-trained model
- ✅ Evaluation harness with LLM-judge, human evaluation and safety results
- ✅ Forgetting / regression check report
- ✅ Quantized model versions with benchmark comparison
- ✅ vLLM / Ollama server with load-test results
- ✅ RAG hybrid over bank policy documents
- ✅ Guardrails and ticketing system
- ✅ Customer chat UI and support-agent dashboard
- ✅ Dockerized deployment with a live link
- ✅ Model on a private Hugging Face repo with a Model Card
- ✅ Cost & ROI analysis
- ✅ GitHub repository with README, setup guide and reproducibility guide
- ✅ 10-minute demo video and slide deck

The finished project should resemble a commercial, self-hosted AI assistant that a bank could deploy without sending customer data to an outside API — demonstrating modern best practices in dataset engineering, parameter-efficient fine-tuning, preference alignment, LLM evaluation, model optimisation, high-performance serving and AI safety.